## Category:

Crypto

## Name:

C4: Analyze the forgotten password

## Message:

**Description:**

You are a computer science professor at a prestigious university, and one of your students, John, has approached you with a unique challenge

**Details:**

John, the student, is in a bit of a predicament. He encrypted his project using a password before an important presentation. Unfortunately, he has forgotten the password and is now unable to access the project.

***Help John recover the password so that he can access his project***

**Notes:**

John leaves some notes:

ko : 8-digit number

ka : 4-digit alphanumeric character

ko : 6-digit alphanumeric character

Order:

0123456789AaBbCcDdEeFfGgHhIiJjKkLlMmNnOoPpQqRrSsTtUuVvWwXxYyZz

Flag format:

CSG_FLAG{***password***}

## Instructions:

First, to see the full image, resize the window to at least 600px. (from 400 -> 600)

```
function setWindowSize() {window.resizeTo(500, 400);}setWindowSize();window.onresiz
```

If the window is resized large enough, they can see the clue “This is top secret!” in reverse.




The pop-up window contains QR codes, and when scanned, the reversed hash values are obtained. *(this is where the clue comes in, to reverse the hash values)*

| var | Key | Md5 | Replaced | Reversed |
|---|---|---|---|---|
| ko | 20231114 | 53578cc8c5c0a6a19c571a1fa6bcab85 | 5d5e8cc8c5c046419c5e141f46bc4b85 | 58b4cb64f141e5c914640c5c8cc8e5d5 |
| ka | fL46 | 8c6e0b255bc7528b7063da96258281a8 | 8c6e0b255bc7528b706flag6258281a8 | 8a1828526galf607b8257cb552b0e6c8 |
| kq | Dec0d3 | d974e538cf501afdfc3986041c5a3993 | d9b4e5z8c2501a2d2cz986041c5az99z | z99za5c140689zc2d2a1052c8z5e4b9d |

Reversed hash values are also defined in the HTA file.

```
var ko="58b4cb64f141e5c914640c5c8cc8e5d5";
var ka="8a1828526galf607b8257cb552b0e6c8";
var kq="z99za5c140689zc2d2a1052c8z5e4b9d";
```

These variables are reversed and replaced by the following functions:

```
function co(o) {
            return (o === dfgtrfd45TYHtredfgr45treRRGTEDFR(ko.replace(/4/g, 'a').replace(/d/g, '3').replace(/e/g, '7')))
}

function ca(a) {
            return (a === dfgtrfd45TYHtredfgr45treRRGTEDFR(ka.replace(/f/g, '3').replace(/l/g, 'd').replace(/g/g, '9')))
}

function cq(q) {
            return (q === dfgtrfd45TYHtredfgr45treRRGTEDFR(kq.replace(/2/g, 'f').replace(/b/g, '7').replace(/z/g, '3')))
}

function dfgtrfd45TYHtredfgr45treRRGTEDFR(string) {
            ver = string.split("").reverse().join("")
            return ver
}
```

You need to analyze keys from MD5 hash values generated by the function by using brute-force manner. Brute force key analysis takes time, but remember the note John left.

```
var ko="58b4cb64f141e5c914640c5c8cc8e5d5"
                    ↓ reverse
          5d5e8cc8c5c046419c5e141f46bc4b85
                    ↓replace d→3, 4→a, e→7
          53578cc8c5c0a6a19c571a1fa6bcab85 → md5 hash of 20231114
                                          ko : 8-digit number  ↑
var ka="8a1828526galf607b8257cb552b0e6c8";
                    ↓ reverse
          8c6e0b255bc7528b706flag6258281a8
                    ↓replace f→3, l→d, g→9
          8c6e0b255bc7528b7063da96258281a8 → md5 hash of fL46
                                ka : 4-digit alphanumeric character ↑
var kq=" z99za5c140689zc2d2a1052c8z5e4b9d ";
                    ↓ reverse
          d9b4e5z8c2501a2d2cz986041c5az99z
                    ↓replace 2→f, b→7, z→3
       d974e538cf501afdfc3986041c5a3993→ md5 hash of Dec0d3
                          kq : 6-digit alphanumeric character ↑
```

You can use John the Ripper to analyze these keys:

```
echo 53578cc8c5c0a6a19c571a1fa6bcab85 > 8-number.txt
john --format=raw-md5 --mask=?d?d?d?d?d?d?d?d 8-number.txt

echo 8c6e0b255bc7528b7063da96258281a8 > 4-digit.txt
john --format=raw-md5 --mask=?a?a?a?a 4-digit.txt

echo d974e538cf501afdfc3986041c5a3993 > 6-digit.txt
john --format=raw-md5 6-digit.txt
```

You can see “20231114” and “fL46” within very short time.

However, John the Ripper needs several minutes (or more) to analyze “Dec0d3”.

(“Dec0d3” password analysis seems to complete faster when no mask is included.)

Let's merge these keys:

| |
|---|
| 20231114 fL46 Dec0d3 |

And John leaves us with another important tip.

| |
|---|
| Order:<br>0123456789AaBbCcDdEeFfGgHhIiJjKkLlMmNnOoPpQqRrSsTtUuVvWwXxYyZz |

Let's rearrange the keys according to this order:

| |
|---|
| 001112233446cDdefL |

The flag is: CSG_FLAG{001112233446cDdefL}

Additional notes:

- The password is based on the combination of 3 strings: 20231114fL46Dec0d3
- And the arrangement of the password is based on this character index array: var nl = [0, 6, 17, 14, 1, 18, 16, 7, 11, 13, 8, 12, 2, 3, 0, 15, 4, 0, 9, 5, 0, 0, 10, 0, 0]; *(the 0 are fillers)*
- For a visual, it's like this:

| 2 | 0 | 2 | 3 | 1 | 1 | 1 | 4 | f | L | 4 | 6 | D | e | c | 0 | d | 3 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 6th | 1st | 7th | 8th | 3rd | 4th | 5th | 10th | 17th | 18th | 11th | 12th | 14th | 16th | 13th | 2nd | 15th | 9th |

From the table, you just have to arrange it in ascending order

| 0 | 0 | 1 | 1 | 1 | 2 | 2 | 3 | 3 | 4 | 4 | 6 | c | D | d | e | f | L |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1st | 2nd | 3rd | 4th | 5th | 6th | 7th | 8th | 9th | 10th | 11th | 12th | 13th | 14th | 15th | 16th | 17th | 18th |

Which is also just an alphabetically arranged order of the string 20231114fL46Dec0d3